

Universidad Tecnológica Nacional

Proyecto Integrador

Análisis sobre algoritmos de ordenamiento.



Materia: Programación I.

Docente Coordinador:

- Alberto, Cortez.

Profesor a cargo de la comisión:

- Julieta Trape.
- Miguel Barrera Oltra

Fecha de Entrega: 09 de junio de 2025

Comisión: 2

Integrantes:

- Dugo, Lucas Nahuel.
- Esper, Amira Yasmin Elizabeth.

Índice

Introducción.	1
Marco teórico.	2
Caso práctico.	10
Conclusión.	16
Bibliografía	17

Introducción.

El **análisis de algoritmos** es una parte fundamental de la teoría de la complejidad computacional, ya que permite obtener una estimación teórica de los recursos necesarios para que un algoritmo resuelva un problema computacional específico. Esta disciplina resulta esencial para medir y comparar la eficiencia de diferentes soluciones en términos de tiempo de ejecución y uso de memoria. Evaluar estos aspectos permite anticipar el comportamiento de un programa frente a entradas de distintos tamaños, y resulta clave al momento de diseñar software escalable, robusto y optimizado.

Además, facilita la elección de enfoques adecuados según las características del problema, sin necesidad de implementar y probar cada solución ante cada modificación del entorno. También permite comparar diversas alternativas, ayudando a seleccionar la más adecuada según los objetivos, las restricciones y el contexto del desarrollo.

Este trabajo se enfoca en el análisis comparativo de algoritmos de ordenamiento, con el objetivo de observar cómo varía su desempeño en función del tamaño de los datos. Se eligió este tema debido a su gran utilidad práctica en múltiples áreas de la programación, donde el manejo eficiente de grandes volúmenes de información es fundamental. A través de un caso práctico implementado en Python, se pretende comprender mejor el comportamiento de los algoritmos seleccionados y reforzar los conceptos teóricos estudiados durante la cursada.

Marco teórico.

Definición de Algoritmo.

Los algoritmos son secuencias ordenadas de instrucciones creadas con el propósito de resolver un problema particular o llevar a cabo una tarea específica. Su funcionamiento se basa en una serie de pasos claramente definidos, en los cuales cada uno cumple una función que contribuye al resultado final.

A continuación, se describen las etapas comunes en la ejecución de un algoritmo:

- **Entrada:** Es el punto de inicio, donde se establecen los datos que el algoritmo va a utilizar. Estas entradas pueden ser simples valores individuales o estructuras más complejas, como listas o matrices.
- **Procesamiento:** En esta fase, el algoritmo aplica operaciones lógicas, matemáticas o de decisión sobre los datos ingresados. Es el núcleo del funcionamiento algorítmico, donde se ejecutan las transformaciones necesarias.
 - ◆ **Toma de decisiones:** Durante el procesamiento, puede ser necesario evaluar condiciones que definan distintos caminos a seguir. Este proceso se lleva a cabo mediante estructuras condicionales que determinan qué acciones ejecutar según el caso.
 - ◆ **Repeticiones (bucles):** Muchos algoritmos requieren ejecutar ciertos pasos varias veces hasta que se cumpla una condición específica. Los bucles permiten repetir acciones de forma eficiente, mejorando el rendimiento y reduciendo la necesidad de instrucciones redundantes.
- **Salida:** Una vez procesadas las entradas, el algoritmo genera una salida. Esta representa la solución al problema planteado o el resultado de la tarea ejecutada.
- **Finalización:** Todo algoritmo debe tener un punto de término claro. Una vez cumplidos todos los pasos y generada la salida, el proceso concluye, garantizando que el algoritmo no se ejecute indefinidamente.

Características de un buen algoritmo.

- **Correcto:** Debe resolver el problema de manera precisa, generando resultados válidos para cualquier conjunto de datos de entrada.
- **Robusto:** Debe manejar situaciones inesperadas sin fallar, asegurando que su ejecución no se interrumpa ante entradas inválidas.
- **Eficiente:** Debe utilizar los recursos de manera óptima, minimizando el tiempo de ejecución y el consumo de memoria.

Los algoritmos son los motores silenciosos de muchas tecnologías y servicios que utilizamos a diario. Tienen una amplia gama de aplicaciones, mejorando la eficacia y personalizando las experiencias en diversos campos. (*¿Qué es un Algoritmo? Definición, Tipos, Aplicación, 2024*). Son la columna vertebral de la tecnología moderna, trabajando entre bastidores para hacer nuestras vidas más fáciles, seguras y agradables, ofreciendo soluciones a medida y optimizando los procesos en diversos sectores. Sus aplicaciones son prácticamente ilimitadas.

Sin embargo, no todos los algoritmos resuelven un problema con la misma eficiencia. Por eso, **el análisis de algoritmos** se convierte en una herramienta fundamental que nos permite estimar el rendimiento de una solución antes de su implementación final, considerando aspectos como el número de operaciones requeridas y la cantidad de memoria utilizada. Esta evaluación ayuda a determinar cuál algoritmo es más adecuado según el tipo de problema, el volumen de datos o las limitaciones del entorno en que se ejecutará.

Enfoques de Análisis de Algoritmos.

Para evaluar la eficiencia de un algoritmo, existen dos enfoques principales:

- **Análisis empírico:** Se basa en la ejecución práctica del código y la medición de tiempos de procesamiento en distintos escenarios, permitiendo observar el rendimiento del algoritmo en condiciones reales.
- **Análisis teórico:** Utiliza modelos matemáticos para estimar la eficiencia sin necesidad de pruebas experimentales. Este método permite clasificar los algoritmos de manera objetiva, independientemente del hardware o las condiciones específicas de ejecución.

Complejidad temporal y complejidad espacial.

La eficiencia temporal y espacial determinan el desempeño de un algoritmo en términos de tiempo de ejecución y consumo de memoria.

- **Eficiencia temporal:** Se refiere a la cantidad de tiempo que un algoritmo tarda en procesar una entrada de datos. Un algoritmo eficiente debe minimizar la cantidad de operaciones realizadas para resolver un problema.
- **Eficiencia espacial:** Representa el consumo de memoria que requiere un algoritmo durante su ejecución. Algunos algoritmos utilizan estructuras auxiliares que pueden aumentar el uso de recursos, lo que debe ser considerado en aplicaciones con restricciones de almacenamiento.

El análisis de algoritmos no solo busca resolver un problema, sino hacerlo de manera rápida y con el menor uso posible de recursos. Un algoritmo puede ser eficiente en términos de tiempo, pero consumir demasiada memoria, o viceversa. Por ello, encontrar un equilibrio entre velocidad y consumo de memoria es clave en el desarrollo de soluciones escalables.

Una vez comprendidos los enfoques de análisis algorítmico, resulta necesario contar con una herramienta formal que permita expresar y comparar la eficiencia de los algoritmos en función del tamaño de entrada. Para ello, se utiliza la notación Big-O, un modelo matemático que describe el orden de crecimiento del tiempo de ejecución frente a entradas crecientes.

Notación Big O.

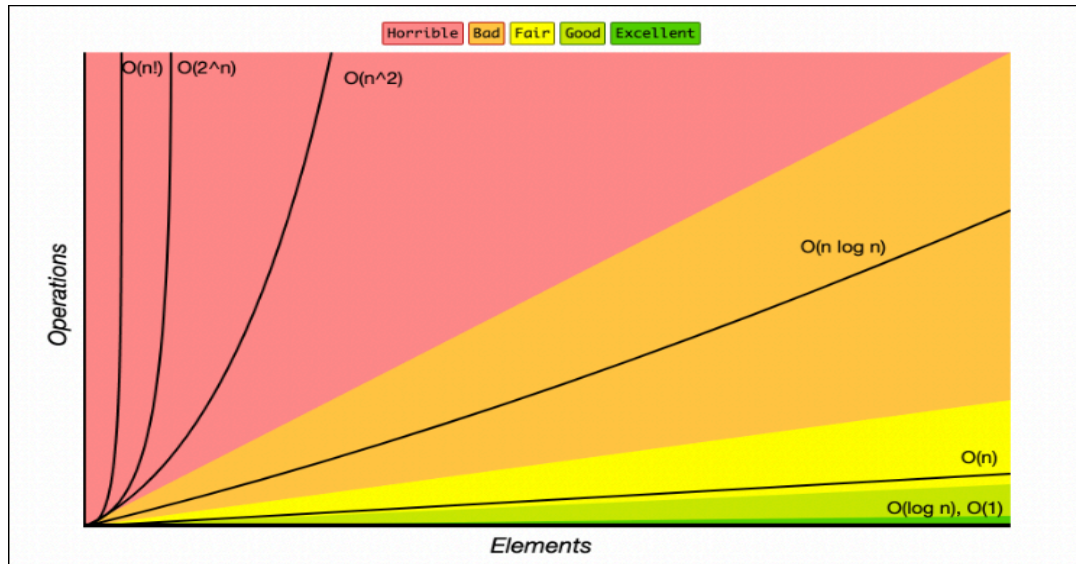
La famosa **Notación Big O** es una métrica que permite expresar de forma general y abstracta, el comportamiento de un algoritmo en relación al tamaño de sus entradas. A través de esta notación, es posible estimar cuánto crecerá el tiempo de ejecución a medida que aumentan los datos procesados, lo que resulta clave para evaluar la escalabilidad y eficiencia de una solución algorítmica.

El significado de la letra “O” en *Big O* es muy importante para comprender el símbolo y aplicarlo al análisis algorítmico. En este contexto, la “O” se refiere al tamaño o tasa de crecimiento de la función que describe la complejidad temporal del algoritmo (The university of science and technology., 2024,). Sirve como símbolo clave que resalta la relación entre el tiempo de ejecución y el volumen de datos, lo que permite a los desarrolladores evaluar rápida y eficientemente la eficiencia relativa de diferentes algoritmos y tomar decisiones informadas sobre su implementación dependiendo del tamaño del problema.

Los algoritmos pueden variar desde **$O(1)$** , que representa un tiempo de ejecución constante e independiente del volumen de datos, hasta **$O(n^2)$** , donde el tiempo crece cuadráticamente con el tamaño de la entrada. Entre estos extremos, existen categorías intermedias como **$O(\log n)$** y **$O(n \log n)$** .

n), que corresponden a algoritmos eficientes en tareas como búsqueda binaria y ordenamiento avanzado.

Figura 1. Gráfico comparativo de complejidades algorítmicas según notación Big-O.



Nota.Guía: Notación Big O - Gráfico de complejidad de tiempo [Gráfico], por Joel Olawale,2024.

[\(Fuente\)](#)

El gráfico de la **Figura 1** representa distintas curvas de complejidad temporal según la notación Big-O. A través de él se visualiza cómo crecen los tiempos de ejecución de los algoritmos en función del tamaño de entrada. Las curvas más planas, como $O(1)$ y $O(\log n)$, corresponden a algoritmos altamente eficientes, mientras que complejidades como $O(n^2)$, $O(2^n)$ y $O(n!)$ presentan un crecimiento abrupto, lo que limita su aplicación en contextos con grandes volúmenes de datos .

A continuación, se presentan las principales complejidades computacionales

Complejidades temporales de la Notación Big O.

Notación Big O	Nombre	Descripción	Ejemplo
$O(1)$	Tiempo constante	El tiempo de ejecución no depende del tamaño de la entrada.	Acceder a un elemento en un array.
$O(\log n)$	Tiempo logarítmico	El tiempo crece lentamente a medida que la entrada aumenta.	Búsqueda binaria.
$O(n)$	Tiempo lineal	El tiempo crece proporcionalmente al tamaño de la entrada.	Recorrer una lista.
$O(n \log n)$	Tiempo lineal logarítmico	Más rápido que cuadrático, pero más lento que lineal.	Algoritmos de ordenamiento eficientes como Merge Sort o Quick Sort.
$O(n^2)$	Tiempo cuadrático	El tiempo crece proporcional al cuadrado del tamaño de la entrada.	Doble bucle anidado, como en Bubble Sort.
$O(2^n)$	Tiempo exponencial	El tiempo se duplica con cada incremento de la entrada. Muy ineficiente.	Recursión en problemas.
$O(n!)$	Tiempo factorial	El tiempo crece factorialmente. Inviabile para entradas grandes.	Algoritmos de fuerza bruta para generar todas las permutaciones

			posibles.
--	--	--	-----------

Una vez comprendido qué es un algoritmo y cómo se mide su eficiencia a través de la notación Big O, es posible aplicar estos conceptos a problemas concretos. Uno de los más conocidos y utilizados es el ordenamiento de datos.

Algoritmos de ordenamiento.

Los algoritmos de ordenamiento permiten organizar registros de una tabla en un orden determinado, de acuerdo con un criterio específico. Son esenciales en múltiples áreas de la computación, como la gestión de bases de datos y el análisis de grandes volúmenes de información. Existen diversos métodos de ordenamiento, que pueden ser interactivos o recursivos, dependiendo de la forma en que se ejecutan. A continuación, se presentan algunos de los algoritmos más utilizados, junto con sus características principales y su implementación en Python.

→ **Bubble Sort (Ordenamiento por burbuja)** : Recorre repetidamente la lista comparando pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Es fácil de entender, pero ineficiente para listas grandes. Ejemplo:

```
def bubble_sort(lista):
    n = len(lista)
    for i in range(n):
        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

→ **Insertion Sort (Ordenamiento por inserción)**: Construye la lista ordenada insertando un elemento a la vez en su posición correcta. Es eficiente para listas pequeñas o casi ordenadas.

```
def insertion_sort(lista):
    for i in range(1, len(lista)):
        actual = lista[i]
        j = i - 1
        while j >= 0 and lista[j] > actual:
            lista[j + 1] = lista[j]
            j -= 1
        lista[j + 1] = actual
```

→ **Selection Sort (Ordenamiento por selección)**: Busca el elemento más pequeño del arreglo y lo coloca en su posición final. Repite este proceso con el resto de la lista. Es simple, pero poco eficiente en la práctica.

```
def selection_sort(lista):
    for i in range(len(lista)):
```

```

min_idx = i
for j in range(i + 1, len(lista)):
    if lista[j] < lista[min_idx]:
        min_idx = j
lista[i], lista[min_idx] = lista[min_idx], lista[i]

```

→ **Merge Sort:** Divide la lista en mitades, las ordena recursivamente y luego las combina en orden. Usa el enfoque "divide y vencerás" y garantiza un buen rendimiento incluso con listas grandes.

```

def merge_sort(lista):
    if len(lista) > 1:
        mid = len(lista) // 2
        izq = lista[:mid]
        der = lista[mid:]

        merge_sort(izq)
        merge_sort(der)

        i = j = k = 0
        while i < len(izq) and j < len(der):
            if izq[i] < der[j]:
                lista[k] = izq[i]
                i += 1
            else:
                lista[k] = der[j]
                j += 1
            k += 1

        while i < len(izq):
            lista[k] = izq[i]
            i += 1
            k += 1
        while j < len(der):
            lista[k] = der[j]
            j += 1
            k += 1

```

→ **Quick Sort:** Selecciona un pivote y reorganiza la lista de modo que los elementos menores queden a un lado y los mayores al otro. Aplica el mismo proceso recursivamente. Es muy rápido en la mayoría de los casos.

```

def quick_sort(lista):
    if len(lista) <= 1:
        return lista
    pivote = lista[0]
    menores = [x for x in lista[1:] if x <= pivote]
    mayores = [x for x in lista[1:] if x > pivote]
    return quick_sort(menores) + [pivote] + quick_sort(mayores)

```

→ **Heap Sort:** Convierte la lista en una estructura llamada heap y extrae sucesivamente el valor mínimo (o máximo). Tiene buen rendimiento y no necesita recursividad.

```

import heapq

def heap_sort(lista):
    heapq.heapify(lista)
    return [heapq.heappop(lista) for _ in range(len(lista))]

```

Los algoritmos de ordenamiento son herramientas fundamentales en la informática, ya que permiten organizar eficientemente grandes volúmenes de datos. Cada algoritmo tiene sus propias ventajas y desventajas en cuanto a velocidad, uso de memoria y facilidad de implementación, por lo que elegir el más adecuado depende del contexto y las necesidades específicas del problema a resolver. Comprender su funcionamiento y rendimiento, especialmente a través del análisis de su complejidad con la notación Big O, es clave para desarrollar soluciones eficientes y escalables. Este conocimiento teórico es la base para abordar con criterio el desarrollo de algoritmos en situaciones prácticas.

Caso práctico.

El presente caso práctico tiene como objetivo analizar y comparar el comportamiento de dos algoritmos clásicos de ordenamiento el Bubble Sort e Insertion Sort desde una doble perspectiva: teórica y empírica. El enfoque adoptado busca no solo identificar cuál de estos algoritmos resulta más eficiente en la práctica, sino también evaluar si el análisis teórico, basado en la notación Big-O, permite anticipar con precisión ese comportamiento.

Las pruebas empíricas se realizaron mediante un único script en Python, que ejecuta de forma secuencial ambos algoritmos sobre las mismas listas generadas aleatoriamente. Para cada tamaño de entrada (500, 1000 y 2000 elementos), el programa mide el tiempo de ejecución de cada algoritmo utilizando `time.time()`, repite la operación tres veces y calcula automáticamente el promedio.

Esta metodología garantiza condiciones equivalentes para ambos algoritmos y permite establecer una comparación válida de su rendimiento práctico.

Se espera verificar que, aun compartiendo la misma complejidad teórica $O(n^2)$, ambos algoritmos presentan diferencias notables en su rendimiento práctico. Asimismo, se busca reforzar el valor del análisis teórico como herramienta para anticipar decisiones informadas en el desarrollo de software eficiente.

Antes de proceder con las pruebas empíricas, se analiza teóricamente el comportamiento de los algoritmos de ordenamiento seleccionados, la deducción de su función temporal $T(n)$. Este análisis permite estimar la cantidad de operaciones que cada algoritmo realiza en función del tamaño de entrada y determinar su eficiencia mediante la notación Big-O.

Si bien ambos algoritmos comparten una complejidad asintótica cuadrática en el peor caso, presentan diferencias estructurales que pueden impactar en su comportamiento práctico. Estas particularidades serán exploradas a continuación.

Análisis teórico de algoritmos de Ordenamiento.

Algoritmo de ordenamiento Bubble Sort.

```
def bubble_sort(lista):  
    n = len(lista)  
    for i in range(n):  
        for j in range(n - 1 - i):  
            if lista[j] > lista[j + 1]:  
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

Descomposición teórica del algoritmo.

1. `n = len(lista)` → 1 operación
2. Primer bucle: `for i in range(n)` → se ejecuta n veces
3. Segundo bucle anidado: `for j in range(n - 1 - i)`

En la peor situación caso en el que la lista esté completamente desordenada podemos representar el total de interacciones como:

$n(n-1)/2$.

4. Dentro del bucle anidado:
 - Comparación: `if lista[j] > lista[j + 1]` → 1 operación
 - Intercambio: `lista[j], lista[j + 1] = lista[j + 1], lista[j]` → hasta 3 operaciones (asignaciones temporales).

Función Temporal (T_n)

$$T(n) = a \cdot n(n - 1)/2 + b \cdot n + c \Rightarrow T(n) = an^2 + bn + c$$

El término de mayor crecimiento eliminando constantes y coeficientes es, n^2 es el que domina la función cuando n crece.

Notación Big-O.

$$T(n) = O(n^2)$$

Algoritmo de Ordenamiento Insertion Sort.

```
def insertion_sort(lista):  
    for i in range(1, len(lista)):  
        actual = lista[i]  
        j = i - 1  
        while j >= 0 and lista[j] > actual:  
            lista[j + 1] = lista[j]  
            j -= 1  
        lista[j + 1] = actual
```

Descomposición teórica del algoritmo.

1. Primer bucle: `for i in range(1, len(lista))` → se ejecuta $n - 1$ veces.
2. En cada iteración:
 - Asignación: `actual = lista[i]` → 1 operación.
 - Comparaciones dentro del while: hasta i veces en el peor caso.
 - Asignaciones de desplazamiento: `lista[j + 1] = lista[j]` → hasta i veces.
 - Reasignación final: `lista[j + 1] = actual` → 1 operación.

En el peor caso que la lista estuviera invertida, el número total de comparaciones y asignaciones se podría expresar:

$$1 + 2 + 3 + \dots + (n - 1) = (n(n - 1))/2$$

Función Temporal (T_n).

$$T(n) = a \cdot n(n - 1)/2 + b \cdot n + c \Rightarrow T(n) = an^2 + bn + c$$

El término de mayor crecimiento eliminando constantes y coeficientes es, n^2 es el que domina la función cuando n crece.

Notación Big-O.

$$T(n) = O(n^2)$$

Ambos algoritmos presentan una complejidad cuadrática en su peor caso. No obstante, Insertion Sort muestra un mejor comportamiento en escenarios favorables, alcanzando una eficiencia de $O(n)$ en el mejor caso, algo que Bubble Sort no logra debido a su estructura de comparación e intercambio fijo.

Ambos algoritmos presentan una complejidad cuadrática en su peor caso. No obstante, Insertion Sort muestra un mejor comportamiento en escenarios favorables, alcanzando una eficiencia de $O(n)$ en el mejor caso, algo que Bubble Sort no logra debido a su estructura de comparación e intercambio fijo.

Análisis Empírico de Algoritmo de Ordenamiento.

Con el objetivo de comprobar si el análisis teórico refleja el comportamiento práctico de los algoritmos Bubble Sort e Insertion Sort, se llevó a cabo una serie de pruebas empíricas implementadas en Python.

Ambos algoritmos fueron aplicados sobre listas de números enteros generadas aleatoriamente, utilizando tamaños de entrada fijos: 500, 1000 y 2000 elementos. Cada prueba se repitió tres veces para cada combinación de algoritmo y tamaño, calculando el promedio de los tiempos de ejecución mediante la función `time.time()`.

Los resultados obtenidos permiten evaluar si el crecimiento del tiempo de ejecución observado se alinea con la complejidad teórica $O(n^2)$ previamente analizada.

Figura 2. Código en python utilizado para medir el tiempo de ejecución promedio de los algoritmos de ordenamiento .

```

Compr_alf_ord_efi.py X
Compr_alf_ord_efi.py > bubble_sort
1 import time # Para medir el tiempo.
2 import random # Para generar listas aleatorias.
3
4 #Algoritmo de ordenamiento Bubble sort.
5 def bubble_sort(lista):
6     n = len(lista)
7     for i in range(n):
8         for j in range(n - 1 - i):
9             if lista[j] > lista[j + 1]:
10                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
11
12 #Algoritmo de ordenamiento Insertion Sort.
13 def insertion_sort(lista):
14     for i in range(1, len(lista)):
15         actual = lista[i]
16         j = i - 1
17         while j >= 0 and lista[j] > actual:
18             lista[j + 1] = lista[j]
19             j -= 1
20         lista[j + 1] = actual
21
22 # Función para medir el tiempo de ejecución
23 def medir_tiempo(algoritmo, datos):
24     tiempos = []
25     for _ in range(3): # Repite la prueba 3 veces
26         lista = datos.copy() # usa una copia para que no este ordenado
27         inicio = time.time()
28         algoritmo(lista)
29         fin = time.time()
30         tiempos.append((fin - inicio) * 1000) # Guarda el tiempo en milisegundos
31     return sum(tiempos) / len(tiempos) # Devuelve el promedio de las ejecuciones
32
33 # * Tamaños de lista que se van a probar
34 tamaños = [500, 1000, 2000]
35
36 # * Imprime encabezado
37 print("Tamaño \tBubble Sort (ms) \tInsertion Sort (ms)")
38
39 # * Ejecuta las pruebas
40 for n in tamaños:
41     lista = [random.randint(1, 10000) for _ in range(n)] # Genera una lista aleatoria
42     t_bubble = medir_tiempo(bubble_sort, lista) # Mide el tiempo de Bubble Sort
43     t_insertion = medir_tiempo(insertion_sort, lista) # Mide el tiempo de Insertion Sort
44     print(f"{n}\t\t{t_bubble:.2f}\t\t{t_insertion:.2f}") # Muestra resultados
  
```

A continuación, se presentan los tiempos promedio de ejecución obtenidos para cada algoritmo, calculados a partir de tres repeticiones por tamaño de entrada.

Figura 3. Imagen de tabla de tiempos de comparación obtenida en python .



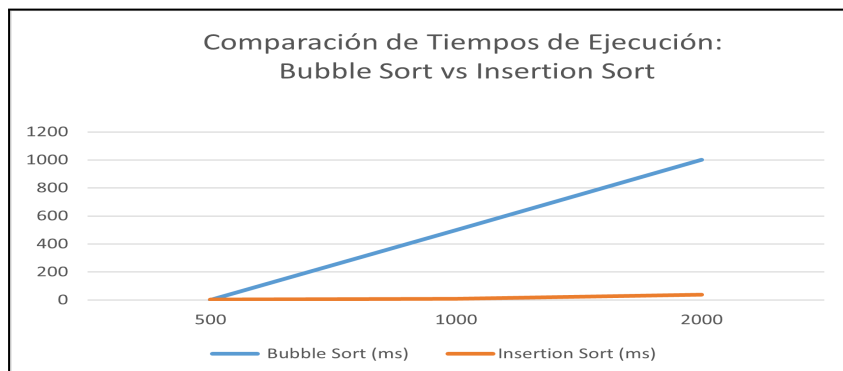
```

PS C:\Users\yases\OneDrive\Desktop\Programacion_1\Proyecto Integrador\Codigos Proyec_int_Analisis de algoritmos de ordenamiento> & C:\Users\yases\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\yases\OneDrive\Desktop\Programacion_1\Proyecto Integrador\Codigos Proyec_int_Analisis de algoritmos de ordenamiento\Compr_alf_ord_efi.py"
Tamaño Bubble Sort (ms) Insertion Sort (ms)
500 5.33 2.18
1000 21.59 9.44
2000 91.69 38.92
PS C:\Users\yases\OneDrive\Desktop\Programacion_1\Proyecto Integrador\Codigos Proyec_int_Analisis de algoritmos de ordenamiento>
  
```

Figura 4. Tiempos promedios de ejecución.

Tamaño	Bubble Sort (ms)	Insertion Sort (ms)
500	5,33	2,18
1000	21,59	9,44
2000	91,69	38,92

Figura 5. El gráfico comparativo.



El gráfico de la **figura 5** representa la comparación visual de los tiempos promedio de ejecución entre Bubble Sort e Insertion Sort para listas de 500, 1000 y 2000 elementos. Se observa un crecimiento más pronunciado en Bubble Sort, lo que evidencia su menor eficiencia práctica.

Los resultados empíricos muestran que, si bien ambos algoritmos tienen la misma complejidad teórica ($O(n^2)$), Insertion Sort presentó un rendimiento significativamente mejor en todos los tamaños de entrada evaluados. Su estructura permite realizar menos operaciones en promedio, especialmente en listas parcialmente ordenadas, lo que se traduce en menores tiempos de ejecución.

Este comportamiento confirma que la eficiencia práctica de un algoritmo no depende únicamente de su clase de complejidad teórica, y refuerza la importancia de complementar el análisis formal con pruebas experimentales.

Conclusión.

El presente proyecto integrador permitió abordar el análisis de algoritmos desde una doble perspectiva: teórica y empírica, aplicando conceptos trabajados durante la cursada para estudiar y comparar el comportamiento de Bubble Sort e Insertion Sort.

A través del análisis teórico, se identificó que ambos algoritmos comparten una complejidad temporal cuadrática en su peor caso $O(n^2)$. Esta estimación permite anticipar un crecimiento significativo en el tiempo de ejecución a medida que aumenta la cantidad de datos, proporcionando una base para evaluar su eficiencia de manera abstracta.

Posteriormente, mediante la implementación práctica en Python y la medición empírica de los tiempos de ejecución sobre listas aleatorias de distintos tamaños, fue posible observar que, a pesar

de tener la misma complejidad teórica, Insertion Sort resultó más eficiente que Bubble Sort en todos los escenarios probados.

Este contraste evidencia que el análisis teórico, si bien es fundamental para clasificar algoritmos y anticipar su comportamiento a gran escala, no siempre refleja con precisión el rendimiento práctico. En consecuencia, se concluye que una evaluación completa de algoritmos debe considerar tanto la complejidad asintótica como las pruebas empíricas, especialmente cuando se busca desarrollar software eficiente y adaptado a contextos específicos.

Bibliografía

- ¿Qué es un algoritmo?* (25 de 04 de 2024). Obtenido de <https://www.datacamp.com/es/blog/what-is-an-algorithm>.
- Aprendiendo del Análisis de Algoritmos en la Programación Competitiva*. (15 de 01 de 2025). Obtenido de <https://beecrowd.com/es/blog-posts/aprendiendo-analisis-algoritmos-programacion-competitiva/>.
- Baudino, G. (09 de 06 de 2020). *Métodos de Ordenamiento*. Obtenido de <https://github.com/gbaudino/MetodosDeOrdenamiento?tab=readme-ov-file>.
- Guía: Notación Big O - Gráfico de complejidad de tiempo*. (01 de 02 de 2024). Obtenido de <https://www.freecodecamp.org/espanol/news/hoja-de-trucos-big-o/>.
- https://www-geeksforgeeks-org.translate.google/what-is-algorithm-and-why-analysis-of-it-is-important/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc. (11 de 11 de 2024).
- ¿Por qué es importante el análisis de algoritmos?*
- Notación Big O y Guía de Complejidad Temporal: Intuición y matemáticas*. (29 de 07 de 2024). Obtenido de <https://www.datacamp.com/es/tutorial/big-o-notation-time-complexity>.
- Notation Big-O*. (04 de 09 de 2024). Obtenido de <https://msmk.university/big-o-notation/>.